Fine-Grained GPU Sharing for Containerised Inference

Architecture overview: the Inference Workload Management Runtime (IWMR) for NVIDIA (MPS) and AMD (CU masking)

Ioannis Kyriakopoulos · Industrial Systems Institute, Athena Research Center · 10 November 2026 · revision 2
Companion document: *Technical Implementation Guide* (commands, scripts and test plan)

1. Purpose and scope

The system runs inference models inside containers that share physical GPUs at a granularity finer than a whole device. All management is performed by a single container per GPU node, the **Inference Workload Management Runtime (IWMR)**. It exposes the only interface to clients and operators, and it admits, places, launches, routes requests to, monitors and recovers the inference containers of its node. No separate scheduler, node agent or monitoring service exists. The same runtime image drives NVIDIA nodes, where sharing relies on the Multi-Process Service (MPS), and AMD nodes, where sharing relies on compute-unit (CU) masking.

Three requirements shape the architecture:

- **Fine-grained shares.** Each container receives a compute ceiling (fraction of SMs or a set of CUs) and a device-memory ceiling, fixed when the container starts.
- **Simple, predictable control.** Placement is deterministic best-fit under a capacity ledger. There is no learning component and no scoring of placements: the same request on the same node state always produces the same placement.
- **Fault containment by design.** Fine-grained sharing does not isolate faults or memory bandwidth. The runtime therefore validates images before they may share a GPU, classifies every exit, drops the workload that caused a fault, and re-places the workloads that were only affected by it.

Out of scope: training workloads, models spanning several GPUs, and coordination across nodes. Each node runs its own IWMR independently; spreading clients over several nodes is left to an ordinary load balancer.

2. Architecture

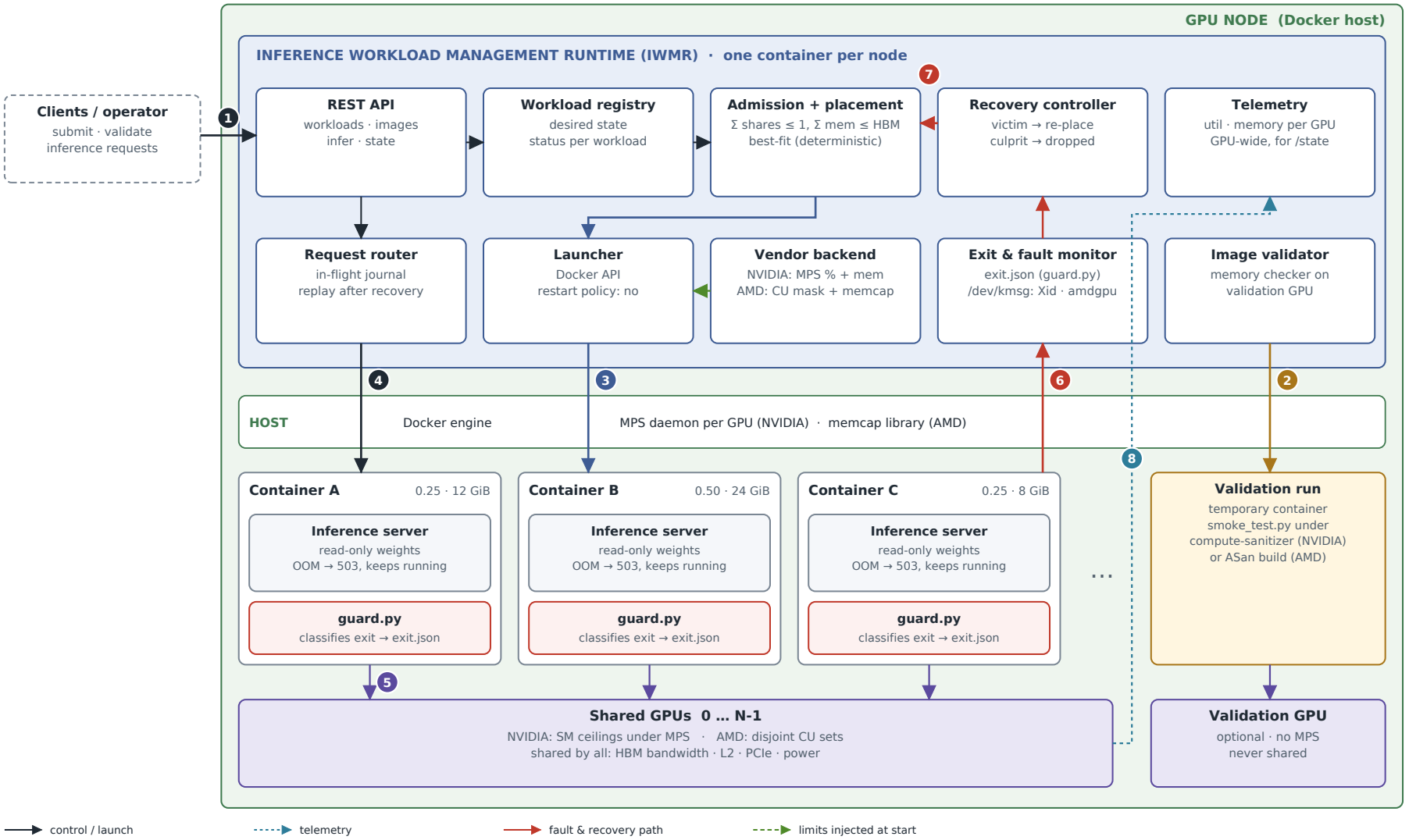


Figure 1. One GPU node. Everything that manages workloads runs inside the IWMR container; the host only provides Docker and, on NVIDIA, the MPS daemons. Numbered arrows correspond to the steps in Section 4.

3. Components

Table 1 lists the components. Ten modules live inside the IWMR container; outside it there are only host services that belong to the GPU driver stack, the inference containers, and the GPUs.

Component	Location	Responsibility	NVIDIA	AMD
REST API	IWMR	Single entry point: submit/stop workloads, validate images, inference requests, node state	—	—
Workload registry	IWMR	Desired state and status (starting, ready, recovering, dropped, failed) of each workload	—	—
Admission + placement	IWMR	Admit only if Σ shares ≤ 1 and Σ (memory cap + context) ≤ device memory; choose the GPU by best-fit	48 clients per MPS server	CU fragmentation checked by backend
Launcher	IWMR	Start and stop inference containers through the host Docker socket, restart policy no	--gpus, --ipc host, MPS pipe	/dev/kfd, /dev/dri/renderD*
Vendor backend	IWMR	Translate a share and memory cap into container settings	MPS thread % + pinned memory limit	HSA_CU_MASK (disjoint CU range) + memcap library
Request router	IWMR	Forward inference requests; journal them until answered; replay after recovery	—	—
Exit & fault monitor	IWMR	Read exit.json on every container exit; attribute kernel-log GPU faults	NVRM Xid records	amdgpu page-fault / reset records
Recovery controller	IWMR	Group exits into incidents; drop the culprit; re-place victims; drain failed GPUs	—	—
Telemetry	IWMR	Sample utilisation and memory per GPU for the node state	nvidia-smi	sysfs (gpu_busy_percent, VRAM)
Image validator	IWMR	Run the image's smoke test under a memory checker on the validation GPU; approve the digest	compute-sanitizer	ASan build of the image
Host services	Host	Container engine and vendor sharing service	Docker; MPS daemon per GPU (systemd)	Docker; memcap library file
Inference container	Host (Docker)	Inference server wrapped by guard.py, which classifies its own exit	same image contract	same image contract

Table 1. Components and their vendor-specific realisation.

4. How the components plug together

The numbered interactions of Figure 1:

1. **Request.** A client or operator calls the REST API: register a workload (image, digest, compute share, memory cap), request image validation, or send an inference request.
2. **Validate.** The validator runs the image's smoke test as a temporary container on the dedicated validation GPU under a memory checker. A clean run approves the image digest. Only approved digests may share a GPU; others can only run alone on a GPU (isolation = exclusive).
3. **Admit, place, launch.** The registry records the workload; admission lists the GPUs with enough remaining share and memory; best-fit picks the one left fullest. The vendor backend produces the container settings and the launcher starts the container through the host Docker engine.
4. **Route.** Inference requests reach containers only through the router, over a private Docker network. Each request stays in the journal until its response returns.

5. **Serve.** The inference server runs kernels within its ceilings. An allocation that hits the memory cap is answered with HTTP 503 and a smaller batch; the router retries after the advised delay.
6. **Report exits.** When a container exits, guard.py has already written the exit class. The monitor reads it, releases the allocation and adds kernel-log attribution where available.
7. **Recover.** The recovery controller applies fixed rules (Section 5) and re-submits affected workloads through admission and placement.
8. **Observe.** Telemetry samples each GPU; the values appear in the node state, and memory use that exceeds the ledger's reservations is flagged as use from outside the runtime.

5. Fault handling

An allocation failure at the memory cap is recoverable inside the container. A fatal fault (illegal address, kernel timeout) invalidates the GPU context; under MPS it also tears down the shared server and ends every client on that GPU. GPU state therefore cannot be saved after the fault. Weights are immutable and reloaded from disk; what must survive is the stream of unanswered requests, which the router holds outside the GPU.

The recovery rules are fixed. The workload identified as the **culprit is dropped**: it is not restarted, its image approval is revoked so it cannot return to a shared GPU until it passes validation again, and its pending requests are failed back to the clients. **Victims** are re-placed at once and their pending requests replayed. Out-of-memory exits are re-placed with a 25 % larger cap; device-level errors drain the GPU.

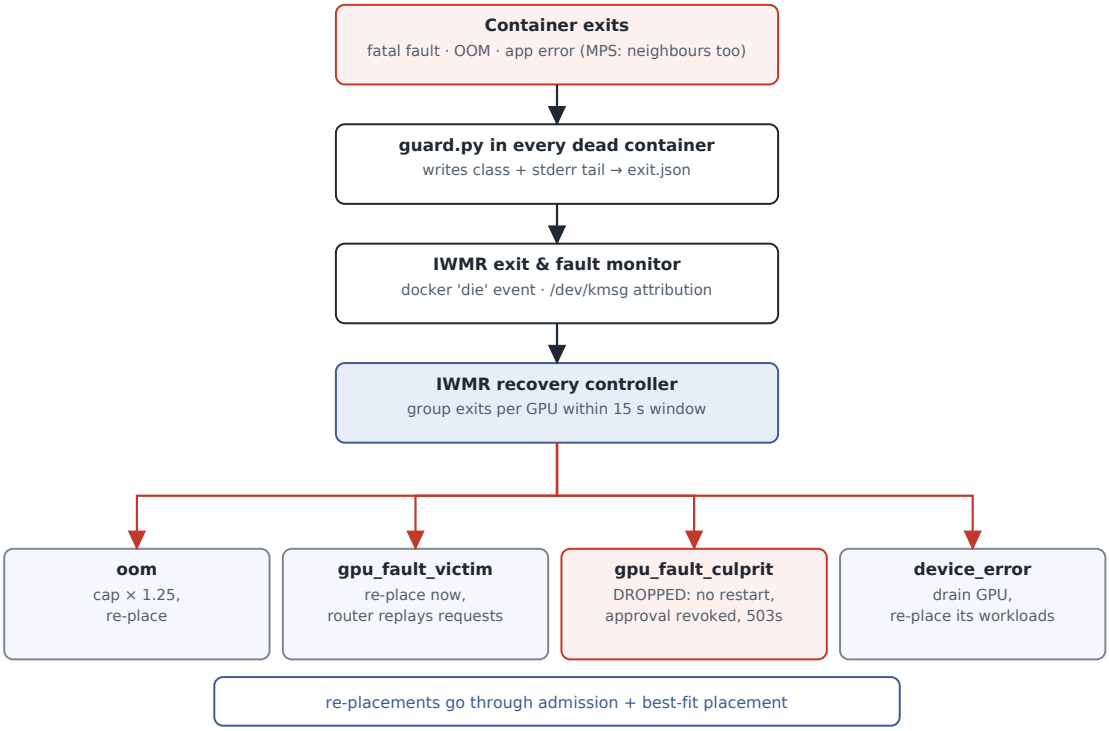


Figure 2. Exit classification and recovery. Exits on one GPU within 15 s form one incident, which separates the culprit from the victims.

6. Isolation provided and its limits

Property	NVIDIA (MPS)	AMD (CU mask)	Runtime measure
Memory protection	Separate address spaces	Separate address spaces	—
Compute share	Ceiling; SMs may overlap	Disjoint CU sets	Ledger keeps Σ shares ≤ 1
Memory capacity	Driver-enforced cap	Library cap (hipMalloc)	Ledger reserves cap + context overhead
Bandwidth / cache	Shared	Shared	Not mitigated; exclusive placement if needed
Fault containment	MPS server: all clients on the GPU	Faulting process; GPU reset affects all	Validation, one MPS daemon per GPU, culprit dropped, request replay
Hard isolation option	MIG (+ MPS inside a slice)	CPX/DPX + NPS partitions (MI300+)	Partitions appear as separate GPUs

Table 2. Isolation by vendor and the runtime measure that compensates where hardware does not.

7. Key design decisions

- **One management container per node.** API, placement, launching, monitoring, recovery, routing, telemetry and validation share one process and one in-memory state, so there is no inter-service protocol to keep consistent. The cost is that the IWMR needs privileged access (Docker socket, kernel log, host PID namespace) and is a single point of failure per node; on restart it removes its containers and starts clean.
- **Ceilings plus admission behave as reservations.** MPS percentages and memory limits are caps. Admitting a workload only when the sums stay within capacity makes them effective reservations, provided nothing outside the IWMR uses the GPUs.
- **Deterministic best-fit placement.** Workloads are packed onto the fullest GPU that still fits, which keeps whole GPUs free for large or exclusive workloads. No smart or learned scheduling is used.
- **Culprits are dropped, not retried.** A workload that faulted is likely to fault again; restarting it would expose new neighbours. Re-admission requires re-validation of the image.
- **One MPS daemon per GPU.** Each GPU becomes its own fault domain on NVIDIA.
- **Validation on a dedicated GPU.** Memory checkers do not run under MPS and are slow, so they gate admission rather than protect production.